

React Hooks Deep Dive

Subject: Web Development Topic: React

Overview

useState, useEffect, useContext, useReducer, custom hooks patterns

State and effects

- useState adds component state; the setter schedules a render and functional updates avoid stale values when the next state depends on the previous one.
- useEffect synchronizes with external systems after rendering; include every reactive value read by the effect in its dependency list.
- Return cleanup from an effect to unsubscribe, clear timers, or cancel obsolete work.

Context and reducers

- useContext reads the nearest provider value and is useful for shared state such as the authenticated user.
- useReducer centralizes transitions as reducer(state, action) and can make multi-step state updates easier to test.
- Keep context values stable when appropriate to avoid unnecessary consumer renders.

Custom hooks and rules

- A custom hook is a function named with the use prefix that composes other hooks and shares stateful logic.
- Call hooks only at the top level of a React component or custom hook, never conditionally or inside loops.
- Hooks share logic, not one shared state instance; each component invocation has its own state.

Common mistakes

- Do not omit effect dependencies to silence lint warnings; restructure the effect or stabilize callbacks when necessary.
- Avoid using effects to calculate values that can be derived during render.
- Use stable keys for list items so React can preserve the intended component identity.